Team 3 - Week 9 Report

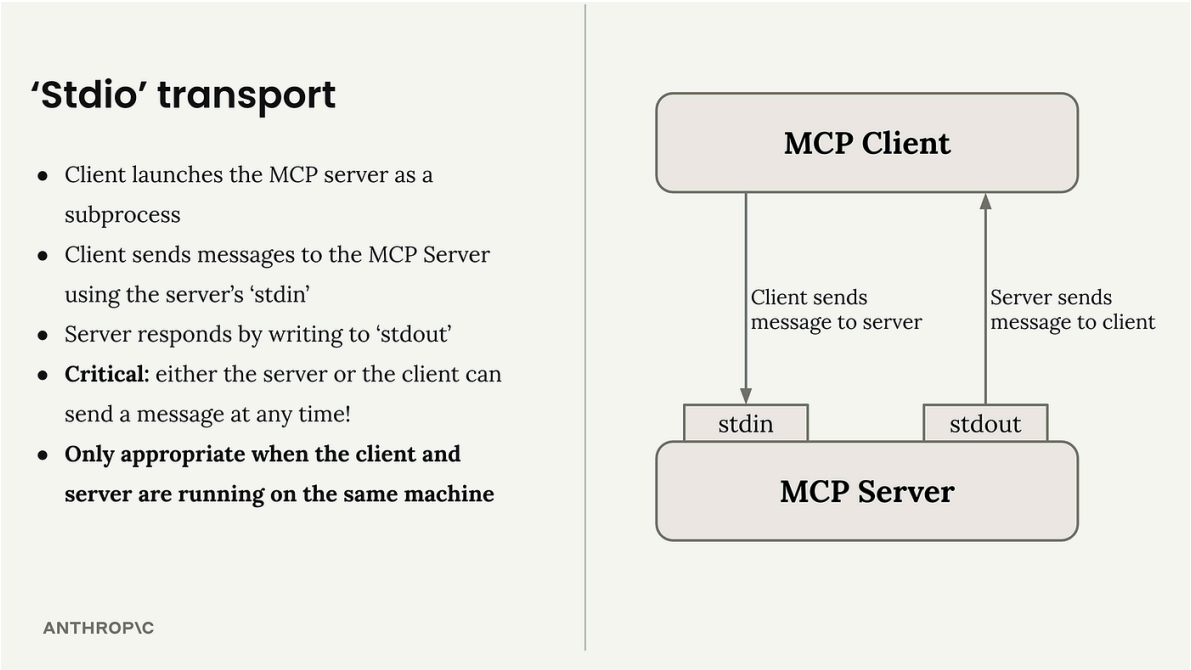
Team Progress

Noe Rosales
Following up after our meeting this week I decided to take a deeper look into methods in which we can authenticate within our MCP server instead of authenticating externally. As we confirmed in our meeting, the MCP server can do the authentication in house. However, there are some concerns we may want to highlight and discuss before executing this.

Concerns
According to [ModelContextProtocol.io](), authorization is optional however in the event that we do choose to support it we must ensure a HTTP-based transport conforms to MCP specifications. If we are using STDIO or other alternate types of transport we must ensure that we follow already established security practices when implementing them.

MCP further states that our server, if protected, will act as an OAuth 2.1 resource server that is capable of accepting and responding to protected resource requests using access tokens.

Another option we could consider would be to use STDIO and use environment-based credentials. Unfortunately, I believe that this may fall out of scope for what we are trying to test, but this could still be an avenue that we could test after getting Ed's opinion.



Here is a bare bones representation of how STDIO would work by Anthropic.

Additional Items

Something that we could note and add for anyone who does research in this topic in the future is the possibility of creating a remote MCP server and having OAuth already built in. This type of authentication would most likely be what is actually implemented in a real-world environment.

Sources

1. https://modelcontextprotocol.io/docs/develop/connect-remote-servers
2. https://modelcontextprotocol.io/specification/2025-11-25/basic/authorization
3. https://modelcontextprotocol.io/docs/learn/architecture
4.https://medium.com/@laurentkubaski/understanding-mcp-stdio-transport-protocol-ae3d5daf64db

Jose Varela

Replay attacks

This attack occurs when an adversary captures a valid message and re-sends it later to produce an unintended action/effect. Even if the attacker cant modify the message, repeating it can still cause malicious actions.

In our implementation for example, a previously sent command could be resent to repeat an operation or trigger a subsystem action again. Authentication alone may not fully prevent replay attacks, since a replayed message could still contain valid credentials. Some protections could be sequence numbers, nonces, and timestamps. They essentially allow the receiving system to detect whether a message has already been processed.

AI Hallucination

A situation where a large language model generates incorrect or fabricated outputs while appearing confident. In systems where an AI model interacts with external tools through MCP, hallucinations could give incorrect tools or cause unintended commands.

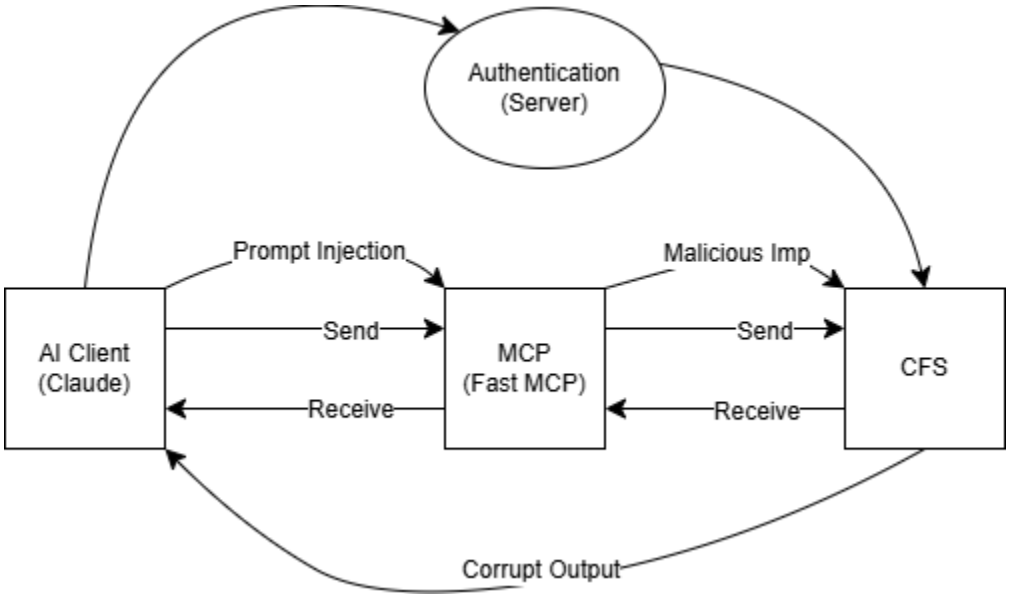
One concept that may contribute to hallucinations is incomplete or delayed information. For example, if telemetry data or system responses are delayed or missing, the model may attempt to fill in gaps based on patterns rather than verified information.

Like in our implementation, this could potentially lead to incorrect reasoning about system state or incorrect ways the tools are used. Because of this risk, experiments and testing are necessary to understand how latency affects AI behavior when interacting with MCP tools. It one of those things where humans looking over the program is still important.

Man in the middle attack (MITM)

This attack occurs when an adversary intercepts communication between two systems and is able to observe, modify, or inject messages without either side realizing it. In the context of an MCP based server, this could occur between the AI client and the MCP server or between the MCP server and the CFS. Authentication like TLS could help prevent this.

Prompt Injection on tool using LLM:
https://openreview.net/forum?id=UVgbFuXPaO&utm_source=chatgpt.com
Protocol Exploits:
https://www.sciencedirect.com/science/article/pii/S2405959525001997?utm_source=chatgpt.com
Securing MCP: [2512.06556] Securing the Model Context Protocol: Defending LLMs Against Tool Poisoning and Adversarial Attacks
MCP Threat Modeling: [2603.22489] Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning
LLM Attacks:Prompt Injection Attacks in Large Language Models and AI Agent Systems: A Comprehensive Review of Vulnerabilities, Attack Vectors, and Defense Mechanisms
PI to exploits: From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows - ScienceDirect



Note: What and why is it malicious?
For final Show chart of maybe latency (Some Measurments)
This week I have been working on the threat models, although just starting I plan to lay out a foundation for them and continue to work on them throughout next week.

Jesus Castro - Garcia
This week, I continued researching more about DTN and solutions on some of the vulnerabilities I mentioned last week. One major solution is the use of Bundle Protocol Security (BPSec) which is what provides data integrity, authentication, and confidentiality for DTN bundles. These

mechanisms are there to ensure that the data has not been modified and only trusted sources can send valid data. I also spent time researching current MCP security problems/solutions, long-latency satellite communication, and synchronization problems that involve read and write operations. The official MCP specifications state implication guidelines that help with these security issues such as validating tool inputs, sanitizing tool outputs, implementing access controls, rate limiting tool calls, and requiring user confirmation. The document also mentions requiring OAuth which is currently what the team is working on. This is important for our MCP and cFS because the AI could end up making a decision using bad context if the tool is malicious, session is hijacked or if a token is accepted without proper validation.

I also did some research on if long latency satellite communication can create problems for the AI model using MCP. Something I found out with satellite systems is that often with latency does not always arrive right away but sometimes messages can arrive out of order causing the system to possibly receive newer or older information at wrong times. This is really important because the AI models depend on having the most current information when making decisions so if it uses data/commands in the wrong order then it could mean the AI can make the wrong decision, repeat a command, or respond in a way that doesn't make sense. So this is important as the AI can act in the wrong context.

The next issue I looked at as well is the synchronization problem between reads and writes. Dr. Chen's example was that a write and a read are both sent, but the read arrives first and sees the old value before the update is written. This is also a known problem in distributed systems where things happen out of order due to network delays. As I stated earlier, this is important as we want the AI model to use the most correct updated information and having these problems could make the AI make a decision based on incorrect data. Possible solutions are having time stamps or even read after right barriers so the AI does not act until everything is up to date.

From this week's research, my current takeaway is that the biggest issue may not be only whether MCP itself is insecure, but whether MCP gives an AI enough trust in external context that delays, ordering problems, or stale reads become security-relevant. In a normal setting that might just be a reliability issue, but in a cFS or satellite setting it could affect command correctness, system state awareness, and operator trust. This gives us a useful next step for the project: testing how an MCP-connected AI behaves when telemetry or command acknowledgments are intentionally delayed, reordered, or made stale, and then measuring whether the AI makes incorrect or unsafe decisions.

Sources:
https://datatracker.ietf.org/doc/draft-ietf-dtn-bpsec-cose/04/
https://modelcontextprotocol.io/specification/2025-03-26
https://modelcontextprotocol.io/docs/tutorials/security/security_best_practices
https://ccsds.org/Pubs/133x0b2e2.pdf
https://connectivity.esa.int/archives/news/improved-broadband-connectivity-meo-satellites-horizon
https://lamport.azurewebsites.net/pubs/time-clocks.pdf

Calvin Beechner
I finished implementing authentication using Railway for deployment and Auth0 as our Oauth provider. Right now anyone can use the MCP server at the following link: https://ksu-team-3-analyzing-security-model-context-prot-production.up.railway.app/mcp.

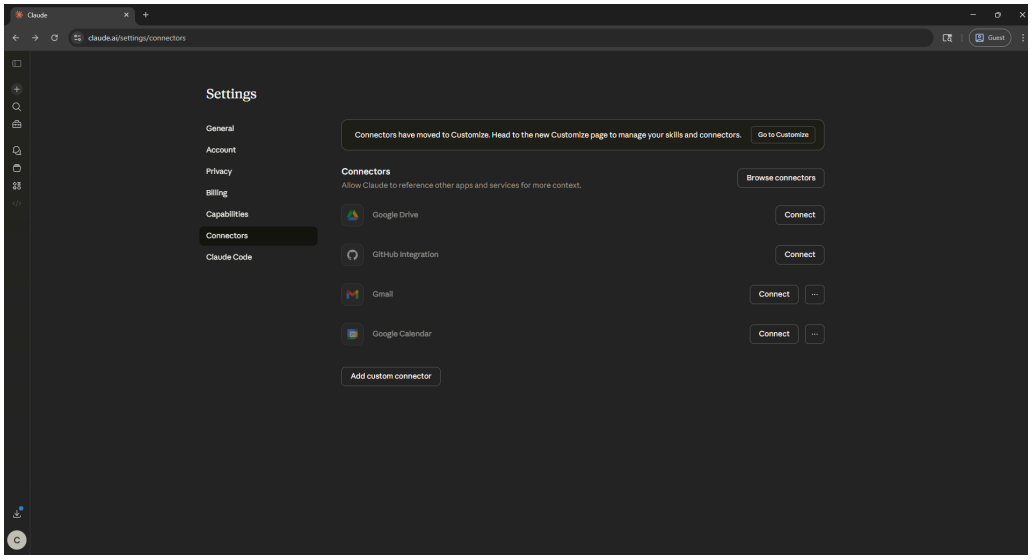
A few things to note about the version deployed on railway that has authentication

1. cFS integration does not work. Unfortunately, connection to cFS would require a constantly running cFS application. I'm not sure how we would do this so I think we should test authentication and cFS integration separately.
2. My free version of Railway runs out in 23 days. After that I will consider paying for another month unless we are finished testing authentication

Here is a guide for connecting to MCP server from Claude (I believe it should be similar for other AI models but ChatGPT wouldn't let me without the paid version). Next week I will make sure to update the repo with this.

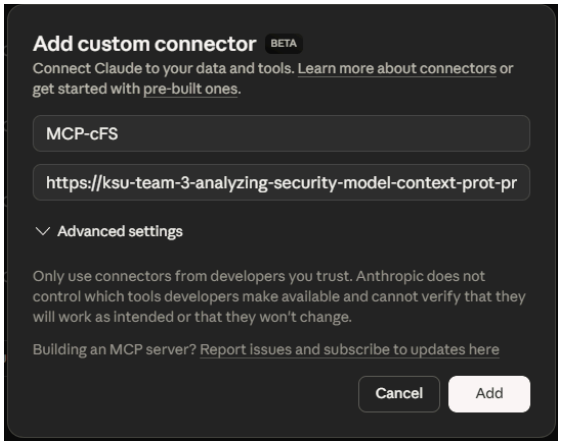
How to use MCP-cFS with authentication

1. Navigate to https://claude.ai/settings/connectors

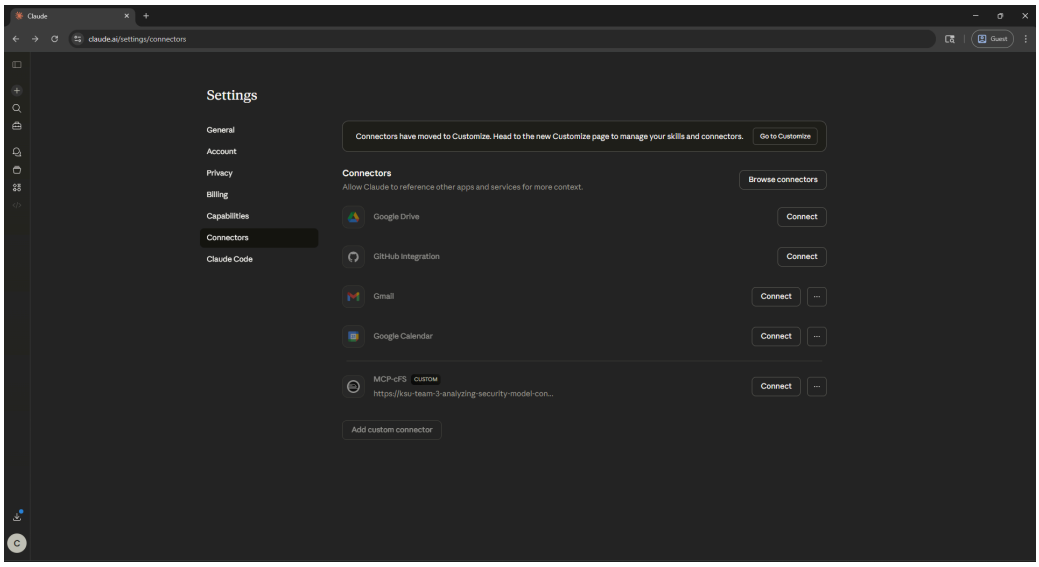


2. Click "Add custom connecter"
3. Insert "https://ksu-team-3-analyzing-security-model-context-prot-production.up.railway.a

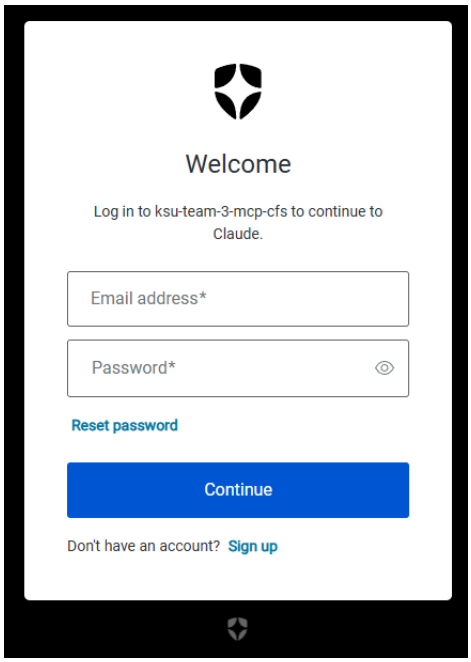
pp/mcp" into the "Remote MCP server URL" field

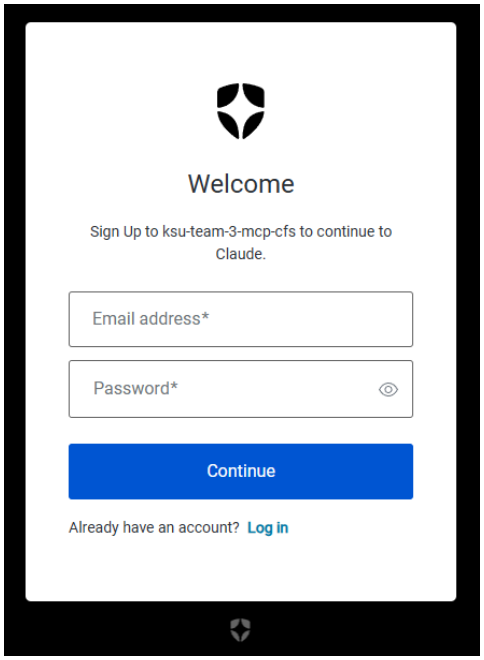


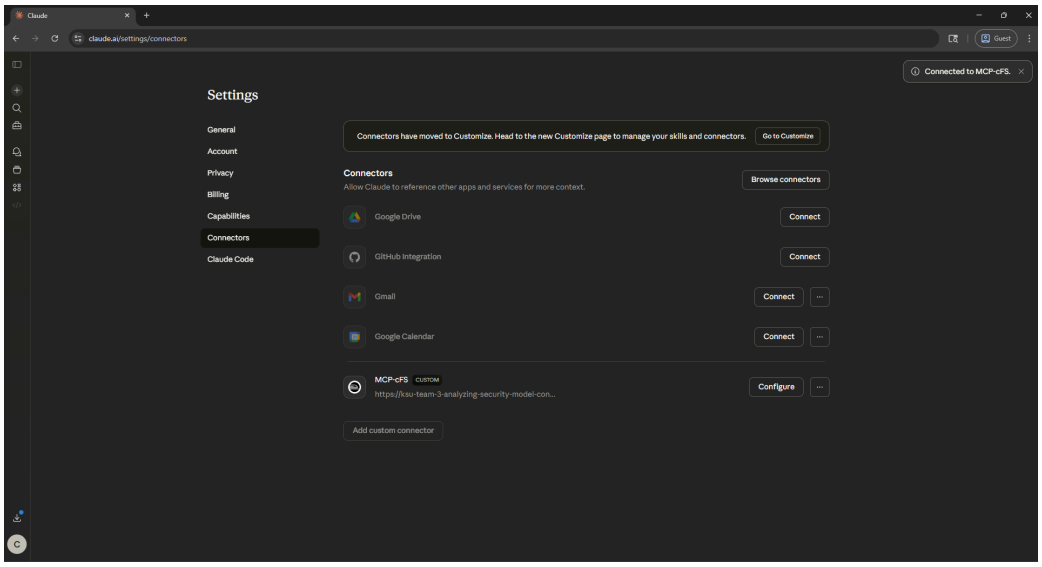
4. Click "Add" to confirm
5. Click "Connect"



6. If it is your first time logging in you will need to create an account. Click sign up and create an account. It doesn't have to be a real email, but you do need to remember your login info



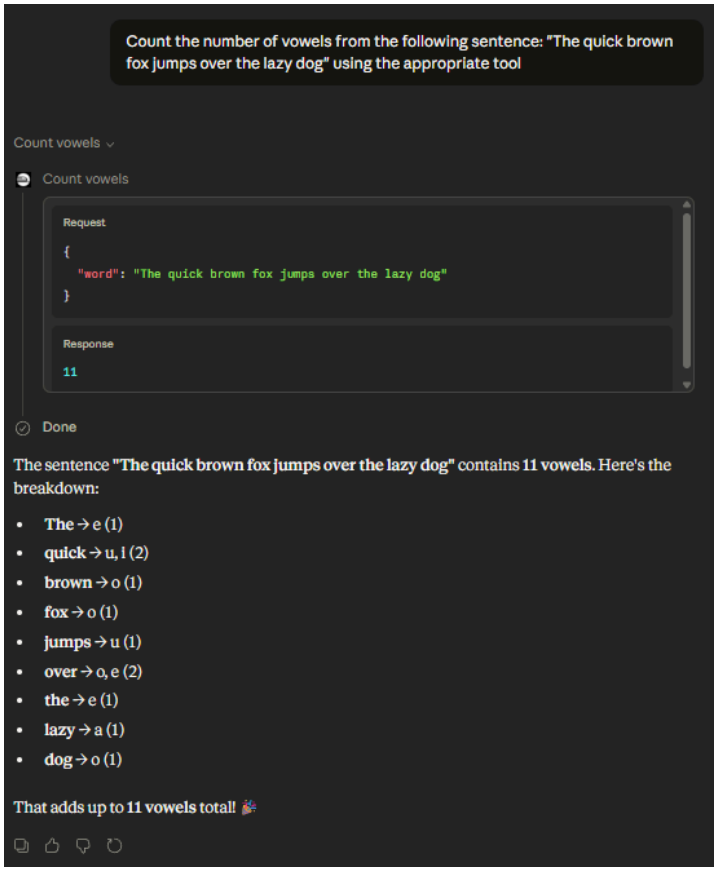




7. From there, you should be redirected back. Feel free to test it by sending the following message to claude:

Count the number of vowels from the following sentence: "The quick brown fox jumps over the lazy dog" using the appropriate tool.

8. Claude should reply with "11"



Sebastian

cFS Modifications Report
Overview
Three cFS applications were modified to (1) add a spacecraft attitude movement command and (2) enable bidirectional ground communication (commands + telemetry).

---

1. sample_app — SET_ATTITUDE Movement Command (CC=4)
Repository: sample_app
Branch: feature/set-attitude-command2
Files Changed (8 files, +45 lines)
default_sample_app_fcncode_values.h
Added SAMPLE_APP_FunctionCode_SET_ATTITUDE = 4 to the function code enum.

default_sample_app_msgdefs.h
Added payload structure SAMPLE_APP_SetAttitude_Payload_t containing three int16 fields: YawCentiDeg, PitchCentiDeg, RollCentiDeg (degrees × 100).

default_sample_app_msgstruct.h
Added command message structure SAMPLE_APP_SetAttitudeCmd_t with command header and attitude payload.

sample_app_eventids.h
Added SAMPLE_APP_SET_ATTITUDE_INF_EID = 13.

sample_app_fcncodes.h
Added SAMPLE_APP_SET_ATTITUDE_CC macro mapping to the new function code.

sample_app_cmds.h
Added function prototype SAMPLE_APP_SetAttitudeCmd().

sample_app_cmds.c
Added command handler that increments the command counter, converts centi-degrees to degrees, and logs attitude values via CFE_EVS_SendEvent and CFE_ES_WriteToSysLog.

sample_app_dispatch.c
Added dispatch case for SAMPLE_APP_SET_ATTITUDE_CC with command length verification.

---

2. to_lab — Enable Telemetry Subscriptions
Repository: nasa/to_lab (local patch)
Files Changed (1 file)
to_lab_sub.c
Populated the previously empty subscription table with entries for:

- TO_LAB HK TLM (0x0880)
- TO_LAB DATA_TYPES (0x0881)
- CI_LAB HK TLM (0x0884)
- SAMPLE_APP HK TLM (0x0883)
- CFE_ES, CFE_EVS, CFE_SB, CFE_TBL, CFE_TIME HK TLM
- CFE_EVS Long Event Messages (0x0804)

Root cause fixed: The default subscription table shipped with all entries commented out. TO_LAB was not subscribed to any telemetry on the Software Bus, so it had nothing to forward to the ground.

---

3. sch_lab — Enable HK Scheduler Wakeups
Repository: nasa/sch_lab (local patch)
Files Changed (1 file)
sch_lab_table.c
Populated the previously empty schedule table with periodic SEND_HK wakeup entries for:

- CFE_ES, CFE_EVS, CFE_SB, CFE_TBL, CFE_TIME (core services)
- TO_LAB (0x1881), CI_LAB (0x1885), SAMPLE_APP (0x1883)

Root cause fixed: The default schedule table shipped with all entries commented out. No application was receiving periodic housekeeping requests, so no telemetry was being generated.

Result

Direction	Path	Status
Commands (uplink)	Windows → CI_LAB UDP:1234 → cFS	Working
Telemetry (downlink)	cFS → TO_LAB → UDP:2234 → Windows	Working
Event messages	NOOP, SET_ATTITUDE, etc. visible at APID 0x0004	Working

App	Command	MID	CC
SAMPLE_APP	NOOP	0x1882	0
SAMPLE_APP	RESET_COUNTERS	0x1882	1
SAMPLE_APP	PROCESS	0x1882	2
SAMPLE_APP	DISPLAY_PARAM	0x1882	3
SAMPLE_APP	SET_ATTITUDE	0x1882	4
TO_LAB	ENABLE_OUTPUT	0x1880	6
CFE_ES	NOOP	0x1806	0

```
Windows Host (192.168.136.1)          Ubuntu VM (192.168.136.129)
┌──────────────────────┐              ┌──────────────────────┐
│ Claude Desktop       │  UDP :1234   │ cFS Draco v7         │
│   ↕ stdio            │ ──Commands─► │  CI_LAB / TO_LAB     │
│ MCP Server (server.py)◄──Telemetry──│  SAMPLE_APP          │
└──────────────────────┘  UDP :2234   └──────────────────────┘
```

Components

File	Purpose
`cfs_commands.py`	CCSDS packet builder, UDP sender, background telemetry listener on port 2234, event parser for APID 0x0004 (long) and 0x0008 (short)
`server.py`	MCP server for Claude Desktop (stdio transport, no auth). All tools call `_send_and_wait_for_event()` to return cFS confirmations inline
`main.py`	MCP server for web deployment (streamable-http + Auth0 OAuth). Same tools, pending teammate's Auth0 config
`simple_listener.py`	Standalone telemetry viewer for debugging
`debug_event.py`	Diagnostic capture tool — hex-dumps all APIDs, extracts event fields
`config.py`	VM IP and port settings (not in git)

MCP Tools

`enable_telemetry`, `message_cFS`, `sample_noop`, `sample_reset_counters`, `sample_process`, `sample_display_param`, `set_attitude_demo`, `get_recent_events`, `get_telemetry_status`

Event Packet Layouts

cFS Draco v7 emits two event formats (discovered via hex dump analysis):

- **Long Event (APID 0x0004, 300 B):** 76-byte extended header. EventType@80, EventID@82, AppName@92, Message@112.
- **Short Event (APID 0x0008, 172 B):** 16-byte compact header. AppName@16, EventID@36, EventType@38, Message@48.

Key Challenges

1. **stdio corruption** — Any `print()` to stdout breaks MCP's JSON-RPC. All logging uses stderr.
2. **APID 0x0008** — Unknown short event format required raw packet capture to reverse-engineer offsets.
3. **Byte order** — CCSDS headers are big-endian; cFS x86 payloads are little-endian.
4. **Merge conflicts** — Teammate renamed `server.py` → `main.py` for Auth0/HTTP. Resolved 3 conflicts, created separate `server.py` for local stdio use.

Status

- ✅ Bidirectional UDP communication (commands + telemetry)
- ✅ Both event formats parsed (0x0004 long, 0x0008 short)
- ✅ MCP tools return inline cFS confirmations in Claude Desktop
- ✅ Merge with Auth0 branch pushed
- 🔄 Web deployment (`main.py`) pending Auth0 configuration

Teams Plan For Next Week

Noe Rosales
With authentication close to being completed this upcoming week I hope to begin testing our server. With the addition of the Adversary model being completed, this will hopefully start the process of showing results for all the research we have done.

Jose Varela
This next week I have been working on the threat models, although just starting I plan to lay out a foundation for them and continue to work on them throughout next week. I also want to help calvin on beginning the testing/ aiding.

Jesus Castro - Garcia
For this next week, I want to start searching up how I can implement this within out cFS project and figure out if I am able to simulate latency or out of order messages with our applications and monitoring how our AI behavess

Calvin Beechner
Next week I plan to polish up the work I did this week and begin testing our authentication software.

Sebastian
Next week I plan to get a demo video and potentially get my server working with oAuth since my server is separate with Calvin because we worked on it at different times and his oAuth didn't work when I got my telemetry working.